

Name: Charmi Gaddale

Roll Number: 23071A0516

AIM: Apply EM algorithm to cluster a set of data stored in a .CSV file. Use the same dataset for clustering using k-Means algorithm. Compare the results of these two algorithms and comment on the quality of clustering.

Dataset used: House Price Prediction dataset

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Preprocessing
from sklearn.preprocessing import StandardScaler

# Clustering algorithms
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture

# Evaluation metric
from sklearn.metrics import silhouette_score
```

```
# Upload dataset
from google.colab import files
uploaded = files.upload()
```

[Choose files](#) Housing.csv

Housing.csv(text/csv) - 29981 bytes, last modified: 19/04/2026 - 100% done
Saving Housing.csv to Housing.csv

```
# Load dataset
df = pd.read_csv("Housing.csv")

# Display first rows
df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	pr
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Dataset information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   price                545 non-null    int64
1   area                545 non-null    int64
2   bedrooms            545 non-null    int64
3   bathrooms           545 non-null    int64
4   stories             545 non-null    int64
5   mainroad            545 non-null    object
6   guestroom          545 non-null    object
7   basement            545 non-null    object
8   hotwaterheating     545 non-null    object
9   airconditioning     545 non-null    object
10  parking             545 non-null    int64
```

```
11 prefarea          545 non-null    object
12 furnishingstatus  545 non-null    object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB
```

```
# Statistical summary
df.describe()
```

	price	area	bedrooms	bathrooms	stories	parking
count	5.450000e+02	545.000000	545.000000	545.000000	545.000000	545.000000
mean	4.766729e+06	5150.541284	2.965138	1.286239	1.805505	0.693578
std	1.870440e+06	2170.141023	0.738064	0.502470	0.867492	0.861586
min	1.750000e+06	1650.000000	1.000000	1.000000	1.000000	0.000000
25%	3.430000e+06	3600.000000	2.000000	1.000000	1.000000	0.000000
50%	4.340000e+06	4600.000000	3.000000	1.000000	2.000000	0.000000
75%	5.740000e+06	6360.000000	3.000000	2.000000	2.000000	1.000000
max	1.330000e+07	16200.000000	6.000000	4.000000	4.000000	3.000000

```
# Select relevant features
data = df[['area', 'price']]
```

```
data.head()
```

	area	price
0	7420	13300000
1	8960	12250000
2	9960	12250000
3	7500	12215000
4	7420	11410000

Next steps: [Generate code with data](#) [New interactive sheet](#)

```
# Standardize the dataset
scaler = StandardScaler()

scaled_data = scaler.fit_transform(data)

scaled_data[:5]
```

```
array([[1.04672629, 4.56636513],
       [1.75700953, 4.00448405],
       [2.21823241, 4.00448405],
       [1.08362412, 3.98575468],
       [1.04672629, 3.55497918]])
```

```
# Apply KMeans clustering
kmeans = KMeans(n_clusters=2, random_state=0)

# Fit and predict clusters
kmeans_labels = kmeans.fit_predict(scaled_data)

# Add labels to dataframe
df["KMeans_Cluster"] = kmeans_labels

df.head()
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	pr
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Apply Gaussian Mixture Model
gmm = GaussianMixture(n_components=2, random_state=0)

# Predict clusters
gmm_labels = gmm.fit_predict(scaled_data)

# Add labels to dataframe
df["GMM_Cluster"] = gmm_labels

df.head()
```

	rice	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	pr
0000	7420	4	2	3	yes	no	no	no	no	yes	2	
0000	8960	4	4	4	yes	no	no	no	no	yes	3	
0000	9960	3	2	2	yes	no	yes	no	no	no	2	
5000	7500	4	2	2	yes	no	yes	no	no	yes	3	
0000	7420	4	1	2	yes	yes	yes	no	no	yes	2	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Silhouette score for KMeans
kmeans_score = silhouette_score(scaled_data, kmeans_labels)

# Silhouette score for GMM
gmm_score = silhouette_score(scaled_data, gmm_labels)

print("KMeans Silhouette Score:", kmeans_score)
print("GMM Silhouette Score:", gmm_score)
```

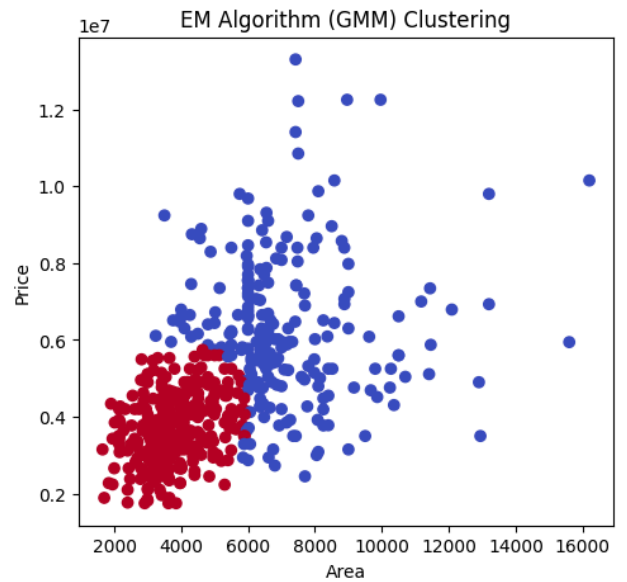
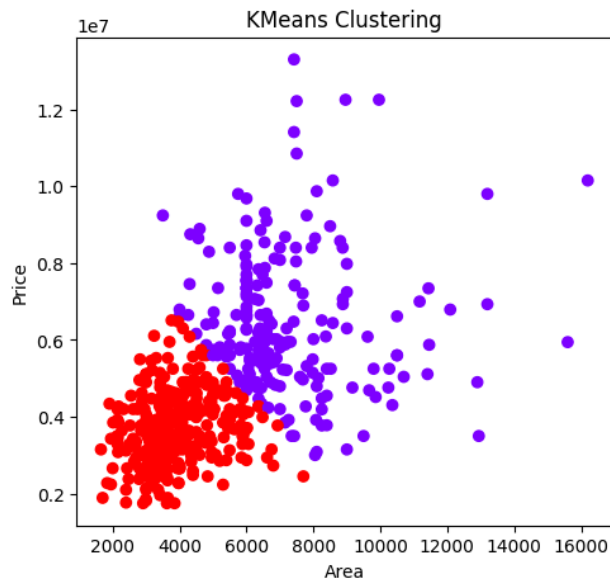
KMeans Silhouette Score: 0.49468992381056137
GMM Silhouette Score: 0.4750628449021969

```
plt.figure(figsize=(12,5))

# KMeans plot
plt.subplot(1,2,1)
plt.scatter(df["area"], df["price"], c=df["KMeans_Cluster"], cmap="rainbow")
plt.title("KMeans Clustering")
plt.xlabel("Area")
plt.ylabel("Price")

# GMM plot
plt.subplot(1,2,2)
plt.scatter(df["area"], df["price"], c=df["GMM_Cluster"], cmap="coolwarm")
plt.title("EM Algorithm (GMM) Clustering")
plt.xlabel("Area")
plt.ylabel("Price")

plt.show()
```



```
print("KMeans Centers:")  
print(kmeans.cluster_centers_)
```

```
KMeans Centers:  
[[ 0.93813269  0.90514117]  
 [-0.57902552 -0.5586628  ]]
```

Inference: The clustering performance of K-Means and the EM algorithm (Gaussian Mixture Model) was compared using the silhouette score and visual plots. The results show that K-Means achieved a slightly higher silhouette score compared to GMM, indicating better cluster separation. Therefore, K-Means performed better for this dataset when clustering based on area and price. However, GMM provides a probabilistic approach and can model more complex cluster shapes.